

Manual Tecnico

Sistema Automatizado de Servicios con Bash

Proyecto Integrador PPS

Ingenieria en Ciberseguridad e Infraestructura de Computo

Programacion para Administracion de Servicios

Repositorio: Hack2-1/proyectodejavi

1. Proposito del sistema

El sistema automatiza tareas comunes de administracion en equipos GNU/Linux mediante scripts Bash. El proyecto integra monitoreo de recursos, respaldos, gestion de usuarios, supervision de servicios, ejecucion remota, monitoreo de red e inventario del sistema.

La solucion se desarrollo con scripts independientes, pero todos comparten configuracion desde config.txt. Esto permite modificar rutas, umbrales, servicios, hosts y parametros sin editar el codigo principal.

Tambien se integro Telegram para recibir avisos en tiempo real dentro del grupo PPS, usando un token guardado fuera del repositorio.

2. Requisitos del proyecto

El documento del proyecto pide siete scripts principales:

- usuarios.sh
- respaldo.sh
- monitoreo.sh
- servicios.sh
- remoto.sh
- red.sh
- inventario.sh

Ademas pide:

- config.txt como archivo compartido.
- Validacion de parametros y entradas.
- Logs con fecha y hora.
- Alertas por Telegram.
- Uso minimo de root.
- trap para interrupciones.
- Manual tecnico y presentacion.
- Repositorio Git con historial de commits.

3. Estructura del repositorio

```
|-- config.txt
|-- configurar-telegram.sh
|-- usuarios.sh
|-- respaldo.sh
|-- monitoreo.sh
|-- servicios.sh
|-- remoto.sh
|-- red.sh
|-- inventario.sh
|-- diagnostico.sh
|-- hosts.txt
|-- crontab.txt
|-- test.sh
|-- setup.sh
|-- README.md
|-- MANUAL_TECNICO.pdf
|-- PRESENTACION_FINAL.pdf
```

diagnostico.sh no es parte de los siete scripts principales, pero se usa para probar remoto.sh porque imprime informacion basica de la maquina donde se ejecuta.

4. Configuración central: config.txt

config.txt guarda los parámetros reutilizables:

- CREDENTIALS_FILE: ruta del archivo seguro de Telegram.
- LOG_DIR: carpeta donde se guardan logs.
- BACKUP_DIR: carpeta de respaldos.
- REPORTS_DIR: carpeta de reportes.
- INVENTORY_DIR: carpeta de inventarios.
- UMBRAL_CPU, UMBRAL_DISCO y UMBRAL_MEMORIA.
- SERVICIOS: servicios que revisa servicios.sh.
- HOSTS_PING y PUERTOS_CRITICOS.
- DIRS_RESPALDO.
- USUARIO_REMOTO, PUERTO_SSH y TIMEOUT_SSH.

También define dos funciones compartidas:

- enviar_telegram: manda mensajes al bot.
- registrar: escribe eventos en logs.

5. Rutas usadas por el sistema

Para no depender de root, el proyecto trabaja con rutas del usuario:

Logs:

`$HOME/.local/state/sistema-servicios/logs`

Reportes:

`$HOME/.local/state/sistema-servicios/reportes`

Inventarios:

`$HOME/.local/state/sistema-servicios/inventarios`

RespalDOS:

`$HOME/respaldos/sistema-servicios`

Credenciales Telegram:

`$HOME/.config/sistema-servicios/telegram.env`

Esto permite ejecutar los scripts desde la terminal de Visual Studio Code sin usar sudo para todo el proyecto.

6. Configuración de Telegram

El bot se configura con:

```
bash configurar-telegram.sh
```

El script solicita:

- Token del bot.
- Chat ID del grupo.

Para grupos de Telegram, el Chat ID normalmente es negativo. En la prueba se usó el grupo PPS con un chat id negativo.

El archivo generado no se sube a GitHub:

```
$HOME/.config/sistema-servicios/telegram.env
```

Prueba de alerta:

```
bash monitoreo.sh -c 0 -d 0 -m 0
```

7. Uso minimo de root

Una buena practica del proyecto es no ejecutar todo como root. Se aplico de la siguiente forma:

- Los logs, reportes, respaldos e inventarios se guardan en HOME.
- usuarios.sh no inicia como root obligatoriamente.
- usuarios.sh usa sudo solo en useradd, passwd, userdel o usermod.
- servicios.sh usa sudo solo si necesita reiniciar servicios.
- setup.sh instala sin root en carpetas de usuario.

Si setup.sh se ejecuta como root, tambien soporta rutas del sistema como /opt, /etc y /var, pero no es obligatorio para la demostracion.

8. usuarios.sh

Funcion:

Permite crear, eliminar, modificar y listar usuarios.

Ejecucion:

bash usuarios.sh

Menu:

1. crear usuario
2. eliminar usuario
3. modificar shell de usuario
4. listar usuarios

Validaciones:

- No permite usuario vacio.
- Valida que el usuario no exista antes de crearlo.
- Valida que el usuario exista antes de eliminarlo o modificarlo.
- Valida que el shell exista.

Salida y evidencia:

- Registra en usuarios.log.
- Manda Telegram por creacion, eliminacion o modificacion.

9. respaldo.sh

Funcion:

Comprime directorios usando tar y genera respaldos .tar.gz.

Ejecucion con directorios de config.txt:

```
bash respaldo.sh
```

Ejecucion indicando directorio:

```
bash respaldo.sh /home/michael/Documentos
```

Proceso:

- Valida que los directorios existan.
- Crea respaldo con tar -czf.
- Verifica que el archivo se pueda listar con tar -tzf.
- Valida que el archivo tenga tamano mayor a cero.
- Elimina respaldos antiguos segun RESPALDO_RETENCION.

Telegram:

Manda archivo, tamano y fecha del respaldo.

10. monitoreo.sh

Funcion:

Lee uso de CPU, disco y memoria.

Ejecucion normal:

bash monitoreo.sh

Ejecucion con umbrales personalizados:

bash monitoreo.sh -c 80 -d 85 -m 90

Forzar alerta para evidencia:

bash monitoreo.sh -c 0 -d 0 -m 0

Funcionamiento:

- CPU se calcula con top o ps.
- Disco se obtiene con df.
- Memoria se obtiene con free.
- Registra cada lectura aunque no haya alerta.
- Manda Telegram si algun valor supera el limite.

11. servicios.sh

Funcion:

Revisa servicios definidos en config.txt o recibidos como argumentos.

Ejecucion:

```
bash servicios.sh
```

```
bash servicios.sh ssh nginx
```

Proceso:

- Valida que systemctl exista.
- Revisa con systemctl is-active.
- Si esta detenido, intenta reiniciar.
- Registra activo, detenido o reiniciado.
- Manda Telegram con el resultado.

Nota:

En una terminal sin permisos, el reinicio requiere sudo. Revisar estado no requiere ejecutar todo el script como root.

12. remoto.sh

Funcion:

Copia un script local a una maquina remota, lo ejecuta por SSH y captura la salida.

Archivo hosts.txt:

192.168.1.72

Ejecucion:

bash remoto.sh diagnostico.sh hosts.txt

Proceso:

- Prueba conexion SSH.
- Copia el script con scp.
- Lo ejecuta en /tmp.
- Captura salida y codigo de salida.
- Borra el archivo temporal.
- Genera reporte general e individual por host.

13. Prueba remota con VM

La VM usada en la demostracion:

IP: 192.168.1.72

Usuario: michael

Sistema: Debian GNU/Linux 13

Primero se acepto la llave SSH nueva:

```
ssh -o StrictHostKeyChecking=accept-new michael@192.168.1.72
```

Luego se ejecuto:

```
bash remoto.sh diagnostico.sh hosts.txt
```

El reporte mostro que el script se ejecuto en debian0, con usuario michael, kernel Debian y recursos de la VM.

14. red.sh

Funcion:

Verifica conectividad y puertos criticos.

Ejecucion con config.txt:

```
bash red.sh
```

Ejecucion con archivo y puertos:

```
bash red.sh -f hosts.txt -p "22 80 443"
```

Clasificacion:

- Accesible: responde ping y tiene todos los puertos abiertos.
- Parcial: responde ping pero algun puerto critico esta cerrado.
- Sin respuesta: no responde ping.

Si hay fallas, manda alerta por Telegram y registra resultados.

15. inventario.sh

Funcion:

Genera un inventario del sistema en texto plano.

Ejecucion:

bash inventario.sh

Datos recopilados:

- Sistema operativo, kernel y hostname.
- Modelo del procesador y nucleos.
- Memoria total, usada y disponible.
- Uso de disco por particion.
- Interfaces de red.
- Usuarios normales.
- Servicios activos.
- Software instalado.

Tambien manda resumen por Telegram con ruta del reporte.

16. diagnostico.sh

Funcion:

Muestra informacion basica del equipo donde se ejecuta.

Ejecucion local:

bash diagnostico.sh

Ejecucion remota:

bash remoto.sh diagnostico.sh hosts.txt

Datos que muestra:

- Fecha, host y usuario.
- Sistema operativo.
- Kernel.
- Nucleos CPU.
- Memoria.
- Disco raiz.
- Tiempo encendido.

17. Cron y automatizacion

El archivo crontab.txt incluye ejemplos para automatizar:

- RespalDOS diarios.
- Monitoreo cada 5 minutos.
- Revision de servicios cada 10 minutos.
- Monitoreo de red cada 15 minutos.
- Inventario diario.
- Diagnostico remoto.

Para usarlo:

`crontab -e`

Despues se copian las lineas necesarias desde crontab.txt.

18. Pruebas del proyecto

Se creo test.sh para validar sin modificar el sistema real.

Ejecucion:

bash test.sh

Pruebas realizadas:

- Existencia de archivos.
- Sintaxis Bash.
- setup.sh en modo check.
- Ayuda de monitoreo y red.
- Ejecucion de diagnostico, inventario y respaldo.
- Red con hosts simulados.

Resultado final:

total: 29

pasadas: 29

falladas: 0

todas las pruebas pasaron

19. Evidencias para la exposicion

Evidencias recomendadas:

1. Telegram configurado correctamente.
2. Alerta de monitoreo forzada con -c 0 -d 0 -m 0.
3. Respaldo generado con ruta, tamaño y fecha.
4. Inventario generado y resumen en Telegram.
5. Alerta de red con host parcial y host sin respuesta.
6. Ejecución remota contra 192.168.1.72.
7. Reporte remoto legible en REPORTS_DIR.
8. test.sh con 29 pruebas pasadas.

20. Errores comunes y solucion

Telegram no llega:

Revisar que TELEGRAM_ENABLED=true en telegram.env, que el token sea del bot correcto y que el chat id sea el negativo del grupo.

getUpdates sale vacio:

El bot no recibio mensajes nuevos, el token no es del bot del grupo o hay un webhook. Usar deleteWebhook y mandar un mensaje nuevo.

SSH falla por llave:

Si la VM cambio, limpiar la llave vieja:

```
ssh-keygen -R 192.168.1.72
```

Servicio no reinicia:

Revisar permisos sudo o ejecutar solo esa accion con privilegios.

21. Estado final

El sistema quedo alineado con la rubrica:

- Siete scripts principales funcionales.
- Configuracion centralizada en config.txt.
- Validacion de entradas.
- Logs con fecha y hora.
- Telegram en eventos importantes.
- Reportes legibles.
- Ejecucion remota real en VM Debian.
- Sin tokens reales dentro del repositorio.
- GitHub actualizado.

Commit principal de scripts:

b7d2438 Actualiza scripts del proyecto integrador

Commit de presentacion:

4615129 Actualiza presentacion final del proyecto